

Genetic Snake

Luca Ghisleni

Matricola 1078852

Corso di Programmazione Evoluta e Competitiva

github.com/lghisleni/GeneticSnake

Sommario

L'obiettivo del progetto è stato realizzare uno snake controllato da una rete neurale e addestrarlo attraverso un algoritmo genetico. Il comportamento dello snake dipende quindi dai parametri della rete, che vengono fatti evolvere nel tempo attraverso selezione, crossover e mutazione. Oltre alla realizzazione dell'ambiente di gioco e del sistema di apprendimento, è stato effettuato un confronto tra sei diverse tecniche di selezione: Tournament, Roulette Wheel, Rank, Stochastic Universal Sampling, Truncation e Sigma Scaling. Il benchmark finale è stato eseguito su cinque run indipendenti per ogni metodo. Tra le configurazioni testate, Truncation Selection ha ottenuto i risultati migliori sia in termini di fitness media sia per quanto riguarda il miglior individuo assoluto trovato.

1 Obiettivo e struttura del progetto

Il progetto parte dall'idea di far imparare autonomamente a uno snake come muoversi all'interno dell'arena. Lo snake riceve alcune informazioni sull'ambiente, le elabora attraverso una rete neurale e decide una delle tre azioni disponibili: continuare dritto, girare a sinistra oppure girare a destra.

L'intero programma è stato sviluppato in Java. La logica è stata suddivisa in componenti separati, in modo da mantenere indipendenti la simulazione del mondo, il comportamento dello snake, il sistema di visione, la rete neurale e l'algoritmo genetico.

L'arena contiene bordi invalicabili e diversi elementi di cibo. Lo snake aumenta di lunghezza quando riesce a mangiare e termina la propria simulazione quando collide con il bordo, con il proprio corpo oppure quando rimane troppo a lungo senza raggiungere nuovo cibo. In particolare è stato introdotto un limite di 900 passi consecutivi senza mangiare. Questa scelta serve a evitare un comportamento che si era rivelato poco utile durante le prime prove: alcuni individui riuscivano a sopravvivere abbastanza a lungo semplicemente muovendosi in cerchio, senza imparare realmente a cercare il cibo.

Per la parte evolutiva è stato utilizzato il framework Watchmaker, mentre la rappresentazione della rete neurale e del DNA è stata gestita direttamente nel progetto. Ogni individuo della popolazione corrisponde quindi a uno snake con una propria rete neurale.

2 Percezione e rete neurale

2.1 Sistema di visione

Lo snake deve prendere una decisione usando soltanto ciò che riesce a percepire dall'ambiente. Per questo è stato realizzato un sistema di visione formato da 32 raggi distribuiti a 360 gradi attorno alla testa. Per ogni raggio vengono ricavate tre informazioni: la distanza dal muro, la presenza del cibo e la presenza del corpo dello snake.

Le distanze vengono trasformate in valori normalizzati nell'intervallo $[0, 1]$: un valore elevato indica che l'elemento rilevato è vicino, mentre un valore basso indica che è lontano o non presente lungo quella direzione. Dal momento che ogni raggio produce tre valori, l'ingresso complessivo della rete è composto da

$$32 \times 3 = 96 \quad (1)$$

valori.

Le intersezioni tra i raggi, il muro e gli oggetti circolari vengono calcolate geometricamente. In questo modo non è necessario controllare punto per punto l'intera lunghezza di ogni raggio. La differenza diventa importante durante l'allenamento, perché la stessa operazione viene ripetuta per migliaia di individui e per un numero molto elevato di passi di simulazione.

2.2 Rete neurale e rappresentazione del DNA

La rete neurale utilizzata ha la seguente struttura:

$$96 \rightarrow 32 \rightarrow 16 \rightarrow 3. \quad (2)$$

I 96 valori prodotti dal sistema di visione vengono inviati al primo strato nascosto, composto da 32 neuroni. Il secondo strato nascosto contiene 16 neuroni, mentre i 3 neuroni finali rappresentano le tre possibili decisioni dello snake: *straight*, *left* e *right*. Negli strati nascosti viene utilizzata la funzione tanh; i valori finali vengono invece trasformati tramite softmax, così da ottenere tre valori confrontabili e scegliere l'azione con risposta maggiore.

Ogni neurone combina gli ingressi ricevuti attraverso pesi e bias. In forma semplificata, il calcolo può essere rappresentato come

$$z_j = b_j + \sum_i x_i w_{ij}, \quad (3)$$

in cui w_{ij} è il peso della connessione e b_j è il bias del neurone. I pesi stabiliscono quanto una determinata informazione influenza il neurone successivo, mentre il bias modifica il livello di attivazione indipendentemente dagli ingressi.

Nel progetto questi valori costituiscono direttamente il DNA dello snake. Il DNA è rappresentato come un vettore `double[]` e contiene tutti i pesi e tutti i bias della rete. Con l'architettura utilizzata sono presenti 3632 pesi e 51 bias, quindi ogni individuo è descritto da un totale di 3683 valori reali.

La popolazione iniziale viene creata assegnando valori casuali compresi tra -1 e 1 . Durante l'evoluzione il crossover combina parti del DNA di due individui, mentre la mutazione modifica casualmente alcuni valori. Il crossover è applicato con probabilità 0.85. La mutazione ordinaria interessa ogni gene con probabilità 0.025 e aggiunge un piccolo rumore gaussiano con intensità 0.25. È presente anche una piccola probabilità, pari a 0.001, di sostituire completamente un gene con un nuovo valore casuale. In questo modo le generazioni successive possono mantenere le caratteristiche utili già emerse, ma allo stesso tempo continuano a esplorare nuove soluzioni.

3 Fitness e processo evolutivo

La parte più delicata è stata definire una fitness che premiasse realmente il comportamento desiderato. Premiare soltanto il tempo di sopravvivenza avrebbe favorito snake capaci di evitare i muri ma non necessariamente capaci di cercare il cibo. Per questo la fitness finale attribuisce maggiore importanza

al numero di cibi mangiati e allo score ottenuto, lasciando comunque un contributo minore al tempo di vita e all'avvicinamento al cibo.

Indicando con f il numero di cibi mangiati, con s lo score, con t il lifetime e con c un valore che misura l'avvicinamento al cibo, la funzione utilizzata può essere riassunta come

$$F = 1500f + 250f^2 + 20s + 0.30t + 250c. \quad (4)$$

Il termine quadratico legato al cibo rende particolarmente vantaggioso riuscire a mangiare più volte. Il termine relativo all'avvicinamento serve soprattutto nelle prime generazioni, quando molti snake non riescono ancora a raggiungere direttamente un elemento di cibo: anche un movimento nella direzione corretta fornisce quindi un piccolo segnale positivo all'evoluzione. In caso di morte contro il muro o di autocollisione, il valore ottenuto viene inoltre ridotto del 10%.

Per ridurre il peso della casualità, ogni individuo viene valutato su due simulazioni con seed prefissati e la fitness utilizzata dall'algoritmo è la media delle due. Inoltre, durante il confronto tra le diverse tecniche di selezione, le run corrispondenti utilizzano gli stessi seed. In questo modo i diversi metodi vengono messi, per quanto possibile, nelle stesse condizioni.

Le sei tecniche di selezione confrontate sono riportate nella Tabella 1.

Tabella 1: Tecniche di selezione utilizzate nel benchmark.

Metodo	Funzionamento
Tournament	Vengono messi in competizione più individui e viene favorito quello con fitness maggiore. Nel progetto la probabilità utilizzata è 0.75.
Roulette Wheel	Ogni individuo ha una probabilità di essere scelto proporzionale alla propria fitness.
Rank	Gli individui vengono ordinati in base alla fitness e la probabilità di selezione dipende dalla posizione ottenuta.
SUS	Stochastic Universal Sampling distribuisce più regolarmente le selezioni lungo la popolazione rispetto alla roulette classica.
Truncation	Solo una parte degli individui migliori viene utilizzata per generare la popolazione successiva. È stata utilizzata una soglia del 50%.
Sigma Scaling	La fitness viene riscalata in funzione della distribuzione dei valori presenti nella popolazione, modificando la pressione selettiva.

Il benchmark finale è stato eseguito con 5 run per ciascun metodo, quindi 30 run complessive. Ogni popolazione contiene 70 individui, di cui 6 vengono mantenuti tramite elitismo. Invece di fissare semplicemente un numero basso di generazioni, è stato impostato un limite massimo di 250 generazioni insieme a un criterio di early stopping. Una run può essere arrestata dopo almeno 60 generazioni se per 40 generazioni consecutive non viene registrato un miglioramento relativo almeno pari allo 0.1%.

Il programma salva tutti i dati necessari: fitness migliore e media per ogni generazione, miglior DNA di ogni run, miglior DNA di ogni metodo e miglior DNA dell'intero esperimento. Inoltre viene salvato il miglior snake ogni 5 generazioni.

4 Risultati

Tutte le 30 run si sono fermate per stagnazione prima di raggiungere il limite massimo di 250 generazioni. La run più breve si è conclusa dopo 66 generazioni, mentre la più lunga è arrivata a 213. La media complessiva è di circa 140 generazioni. Questo comportamento mostra che il limite massimo era sufficientemente ampio e che l'early stopping ha evitato di continuare a calcolare generazioni che non stavano più producendo miglioramenti significativi.

Tabella 2: Risultati aggregati delle cinque run per ogni metodo.

Metodo	Media best	Dev. std.	Best assoluto	Gen. medie
Truncation	240276.61	31050.37	279355.24	129.2
Sigma Scaling	220489.21	21481.57	234858.94	146.8
SUS	210682.16	50764.38	275810.55	159.4
Rank	198999.30	31222.10	224793.50	104.8
Roulette Wheel	188810.45	48793.92	240163.55	140.8
Tournament	185706.65	32584.69	239161.31	159.8

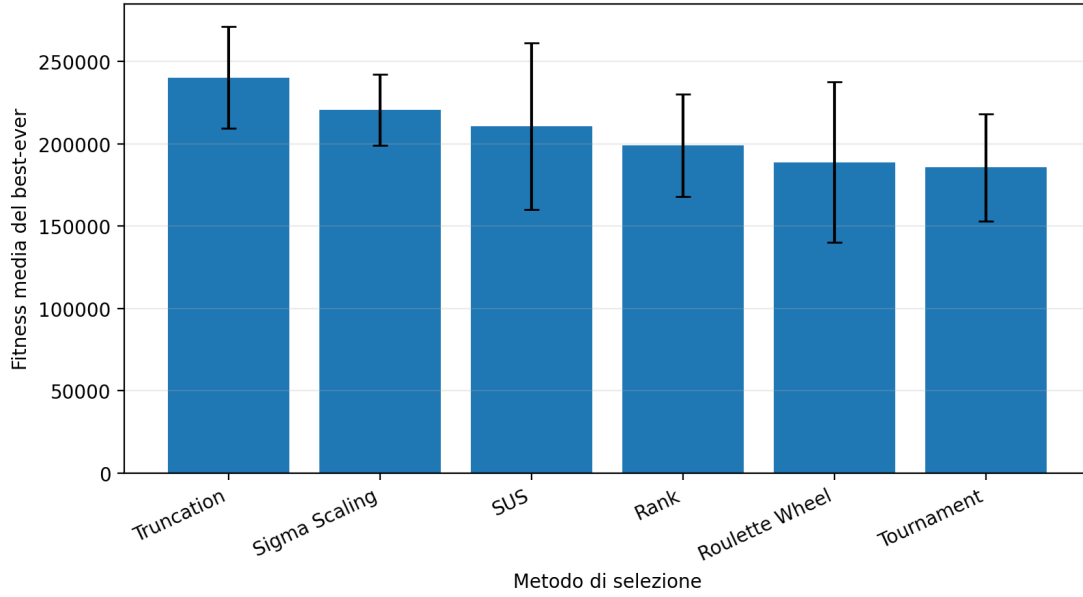


Figura 1: Fitness *best-ever* media ottenuta dai sei metodi.

Il risultato migliore è stato ottenuto da Truncation Selection, con una fitness *best-ever* media pari a 240276.61. La mediana è 231917.47 e la deviazione standard è 31050.37. Il miglior snake dell'intero benchmark appartiene allo stesso metodo ed è stato trovato nella terza run, alla generazione 78, con fitness 279355.24.

Il confronto non va però letto soltanto guardando il record assoluto. SUS, per esempio, ha prodotto uno snake con fitness 275810.55, quindi molto vicino al record di Truncation, ma presenta anche la deviazione standard più alta tra i metodi testati. Questo significa che le sue prestazioni sono state molto diverse da una run all'altra. Sigma Scaling si comporta in modo opposto: non raggiunge il record assoluto più alto, ma ottiene la seconda media migliore e la deviazione standard più bassa, mostrando risultati più regolari.

Truncation risulta quindi interessante perché unisce tre aspetti: ottiene la media più alta, mantiene una variabilità inferiore a metodi come SUS e Roulette e produce anche il miglior individuo assoluto. Nel benchmark effettuato è perciò il metodo che ha dato il risultato complessivamente più convincente.

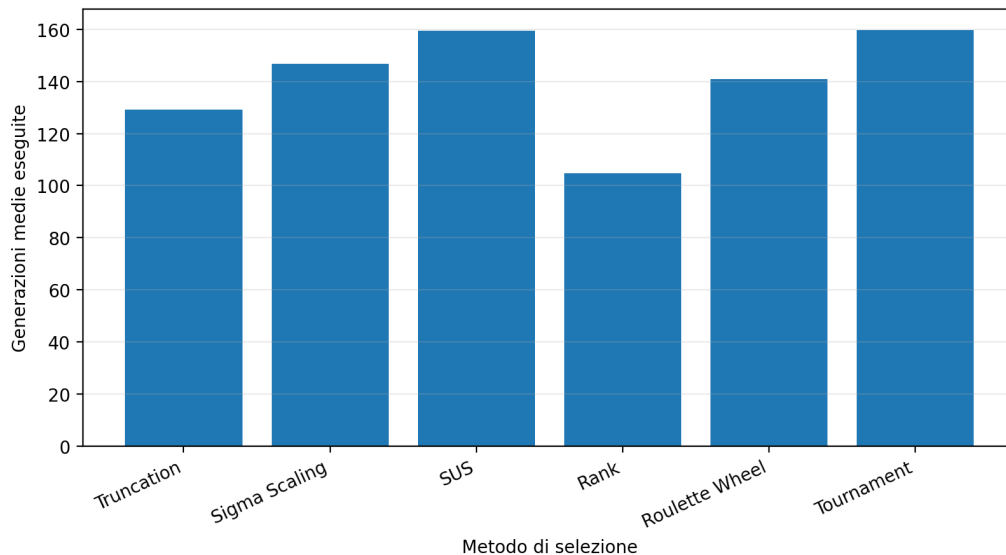


Figura 2: Numero medio di generazioni eseguite prima dell'arresto per stagnazione.

Un altro dato utile è il numero di generazioni necessario prima dello stop. Rank è il metodo che termina mediamente prima, con 104.8 generazioni, ma raggiunge una fitness inferiore rispetto ai primi tre metodi della classifica. Truncation arriva in media a 129.2 generazioni, mentre SUS e Tournament richiedono circa 159 generazioni. Questo mostra che terminare prima non significa necessariamente aver trovato una soluzione migliore: nel caso di Rank la convergenza è più rapida, ma avviene verso valori di fitness meno elevati.

5 Conclusioni

Il progetto ha permesso di realizzare un sistema completo in cui il comportamento dello snake emerge dall'evoluzione dei parametri della rete neurale. A partire dalle informazioni ottenute dal sistema di visione, ogni individuo prende autonomamente le proprie decisioni di movimento, mentre l'algoritmo genetico modifica progressivamente pesi e bias nel corso delle generazioni.

Il confronto sperimentale ha mostrato che la tecnica di selezione influenza in modo significativo la qualità delle soluzioni ottenute. Tra i sei metodi analizzati, Truncation Selection è risultato complessivamente il più efficace: ha ottenuto la fitness media migliore sulle cinque run e ha prodotto anche il miglior individuo assoluto dell'intero benchmark.

Una possibile spiegazione è legata alla maggiore pressione selettiva introdotta da Truncation. Questo metodo permette infatti soltanto alla parte migliore della popolazione di contribuire alla generazione successiva. Quando emerge una configurazione della rete neurale particolarmente efficace, le sue caratteristiche hanno quindi una maggiore probabilità di essere mantenute e combinate nelle generazioni successive. Nel problema considerato questo comportamento sembra essere stato vantaggioso, perché ha consentito di consolidare rapidamente le soluzioni migliori invece di distribuire la riproduzione anche tra individui con prestazioni più basse.

La pressione selettiva elevata potrebbe, in generale, ridurre troppo rapidamente la diversità della popolazione. Nel progetto, tuttavia, crossover e mutazione continuano a introdurre variazioni nei nuovi individui, permettendo di mantenere una certa capacità di esplorazione. Truncation sembra quindi aver raggiunto un buon equilibrio tra la conservazione delle soluzioni già efficaci e la possibilità di generarne di nuove.

Gli altri metodi hanno comunque mostrato caratteristiche interessanti. Sigma Scaling ha prodotto risultati particolarmente regolari, mentre SUS è riuscito in alcune run a raggiungere valori molto

elevati, mostrando però una maggiore variabilità. Rank ha invece raggiunto più rapidamente la condizione di stagnazione, senza ottenere gli stessi livelli di prestazione di Truncation.

L'early stopping ha infine permesso di interrompere automaticamente le run quando l'evoluzione non produceva più miglioramenti significativi. Tutte le 30 esecuzioni si sono infatti concluse prima del limite massimo di 250 generazioni, evitando calcoli aggiuntivi non necessari.

Sulla base dei risultati ottenuti, Truncation Selection può quindi essere considerata la tecnica di selezione più adatta alla configurazione utilizzata nel progetto. Questo non significa che sia in assoluto il metodo migliore per qualsiasi problema evolutivo, ma che, con la rete neurale, la rappresentazione genetica e i parametri adottati in questo esperimento, è stata la strategia capace di produrre le soluzioni complessivamente migliori.